BrainAI

# ProSuite Industrial

INDUSTRIAL ENCRYPTION SUITE

**PRODUCT DOCUMENTATION**

BrainAI (Inhaber: Sascha Alexander Köhne)

| BrainAI Inhaber: Sascha Alexander Köhne | PROSUITE INDUSTRIAL | 🟢 PUBLIC CLASSIFICATION |
|---|---|---|
| Core Architecture & Development | **ProEDC System Integration Guide** | Access Level: Public / Beta |

---

## Revision History

| Version | Date | Author | Status | Description |
|---|---|---|---|---|
| 1.0.0 | 2025-01-01 | S. Köhne | Stable | Initial Release of Wrapper Suite |
| 2.0.0 | 2026-02-27 | S. Köhne | Stable | Full API Integration (Hashing, KeyGen, Crypto) |

# 1 Product Overview

The **ProEDC Wrapper Suite** serves as the bridge between high-level applications and the native ProEDC platform. It provides access to the full functional spectrum of the suite: from hardware-level entropy generation to high-speed hashing and hybrid file encryption.

**Core Components of the Wrapper API:**

- **ProEDC High-Level API:** Simple methods for file operations and initialization.
- **ProKey Integration:** Direct access to the native entropy source for key generation.
- **ProHash Integration:** Low-level access to the hashing context for data streams and files.
- **ProED Core:** Granular control over encryption contexts and IV handling.

# 2 Common Data Structures & Status

To ensure binary compatibility, all wrappers map the native C structures exactly.

### 2.1 ProEDC_Status (Error Handling)

| Constant | Value | Description |
|---|---|---|
| `SUCCESS` | 0 | Operation completed successfully. |
| `ERR_PARAM` | -1 | Invalid parameters (e.g., incorrect key length). |
| `ERR_LIMIT_REACHED` | -7 | **License Stop:** The volume limit of the edition has been reached. |

### 2.2 ProHash_Ctx (Hashing Context)

Used to calculate hashes over large amounts of data in chunks. The structure includes the internal state (`belt`), the byte counter (`count`), and the buffer (`buffer`).

## 3 C# (.NET) – Deep Integration

The C# wrapper utilizes `unsafe` pointer arithmetic for maximum performance and provides direct access to all core modules.

### 3.1 File Encryption & Hashing

```
using ProEDC;

// 1. Initialize the suite (License & driver check)
ProNative.ProEDC_Init();

// 2. Generate a new 256-bit key (via ProKey)
byte[] myKey = new byte[32];
fixed (byte* pKey = myKey) {
    ProNative.ProKey_Generate(pKey, (UIntPtr)32);
}

// 3. Encrypt file (with integrated license check)
ProEDC_Status st = ProNative.ProEDC_EncryptFile("source.dat", "enc.da
if (st == ProEDC_Status.ERR_LIMIT_REACHED) HandleUpgrade();

// 4. Use ProHash context for stream hashing
ProHash_Ctx hCtx = ProNative.CreateHashContext();
ProNative.prohash_init(&hCtx);
// ... prohash_update & prohash_final ...
```

## 4 Java (JNA) – Enterprise Standard

The Java implementation uses JNA structures with strict memory alignment (4-byte for context, 8-byte for hash).

### 4.1 Automated File Operations

```
import com.brainai.proedc.ProNative;

// High-Level Client uses the proedc.dll / .so
// Generates keys, calculates hashes, and processes files
byte[] key = ProNative.Client.generateKey(32); // Uses ProKey
String fileHash = ProNative.Client.calculateHash(someData); // Uses P

int status = ProNative.Client.encryptFile("input.db", "output.pro", "
```

## 5 Python – Automation & Analysis

The Python wrapper maps the `ctypes` structures for `ProEDContext` and `ProHashCtx` to achieve native performance in scripts.

### 5.1 Full Functional Example

```python
from proedc import ProEDC

client = ProEDC()

# 1. Verify file integrity
hash_val = client.calculate_hash(b"System-Integrity-Check") # ProHash

# 2. Key Management
new_key = client.generate_key(32) # ProKey

# 3. Commercial File API
status = client.encrypt_file("config.json", "config.enc", "master.key
if status == 0:
    print(f"Encryption completed. Hash: {hash_val}")
```

## 6 JavaScript (Node.js) – Backend Security

Utilizes `ffi-napi` and `ref-struct` to manage the 72-byte native contexts directly in Node.js memory.

### 6.1 Integration

```
const proedc = require('./proedc');

// Initialize the engine (ProEDC_Init)
const key = proedc.generateKey(32); // Hardware entropy
const hash = proedc.calculateHash("Data Stream Content"); // ProHash-

// Status codes are returned directly as integers
const res = proedc.encryptFile('archive.tar', 'archive.proedc', 'key.
```

## 7 VB.NET – OT & Legacy Support

The VB implementation uses `MarshalAs` attributes to ensure the fixed array sizes of the native structures (`SizeConst:=8` for state, `SizeConst:=64` for buffer).

### 7.1 Example

```
Imports ProEDC

' Verify suite and version
ProNative.ProEDC_Init()
Dim ver As String = ProNative.GetVersionString()

' File access via Commercial API
Dim st As ProEDCStatus = ProNative.ProEDC_EncryptFile("data.bin", "da
```

## 8 Best Practices for Wrapper Developers

- **Initialization:** Call `ProEDC_Init()` once at application startup. This ensures that license checks and hardware bypass checks (PRO Edition) are executed correctly.
- **License Handling:** Always implement a check for `ERR_LIMIT_REACHED` (-7). This is not a technical failure, but an administrative event (TRIAL/INDUSTRIAL volume exhausted).
- **Key Protection:** Use the `ProKey_Generate` methods of the wrappers instead of using your own random functions to guarantee the full entropy of the ProSuite.

---

*Note: This documentation describes the complete API interface. For details on the internal implementation of the cryptographic cores, please consult the internal specification documents.*

---

| | | |
|---|---|---|
| **© 2026 Sascha Köhne** | ✉️ **Contact:** **koehne83@googlemail.com** | 🛡️ **ProTU-Verified** |
| CEO / System Architect | BrainAI Inhaber : Sascha Alexander Köhne | We don't need **BRUTEFORCE**, we know **PHYSICS** |